Embedded System

Francesco De Patre – 382760

Introduction

The fourth lab focuses on configuring and using output and debugging mechanisms.

Two distinct approaches were used: printing directly to the terminal using the printk() function and using the built-in logging module.

Unlike traditional programming, in embedded systems output operations are not just a debugging tool, but represent a tradeoff between visibility into the system state and consumption of limited resources, such as memory and execution time.

Methodology

The workshop was conducted using the Nordic Developer Academy material as the primary reference, supplementing the instructions with practical tests on the device and direct observation of the serial output.

The approach adopted was iterative: after an initial execution of the provided examples, changes were made to the code and system configuration to analyze the behavior of the different output methods.

In particular, we compared the effects of using printk() versus the logging module, varying the logging levels and observing the differences in readability and control.

Debugging was done primarily via serial terminal, using the device's UART output.

LAB Section

Technical objective

The goal of the lab is to understand output mechanisms in embedded systems by comparing the direct approach via printk() with the advanced logging system provided by Zephyr, and analyzing the implications in terms of efficiency and resource management.

What was done during the laboratory

During the lab, simple terminal printing examples were implemented using printk(), followed by using the logging module to produce structured output.

Subsequently, the logger configuration parameters were modified to control the level of verbosity of the messages, even limiting the output to error messages only via configuration in the project file.

What can be understood

This lab highlighted how output management is a non-trivial aspect in embedded systems.

In particular, the printk() function represents a simple and immediate way, but it is not very flexible and not suitable for complex projects, while the logging module introduces an abstraction level that allows you to filter messages by severity level, enable/disable logging without modifying the code, improve readability through formatting and timestamps.

It also emerged that logging has an impact on system resources (memory and execution time), making it necessary to balance the amount of information and the overhead introduced.

This reflects a fundamental aspect of embedded development: every functionality, even seemingly simple ones like printing to a terminal, must be evaluated in terms of computational cost.

Problems encountered

One of the main problems encountered concerns the configuration of the logger, which is not immediate and requires the correct modification of the parameters in the project configuration file.

Additionally, it was necessary to understand the difference between compile-time and runtime configuration, as some changes would not take effect without a full recompilation (pristine build).

Possible improvements

One possible improvement is to use dynamic logging configurations to adapt the level of detail based on the development stage.

Finally, integrating logging with more advanced debugging tools could further improve the analysis of system behavior.

Evidence

The following images refer to the output of the same function (calculating the factorial of a number I = 1...10) respectively:

- Example 1 (Simple printing with the printk function)
- Example 2 (printing with the logger module)
- Example 3 (printing with the logger module with custom configuration)

USB-DEV
16BEDDB16139A2C3

TERMINAL SETTINGS ABOUT

SERIAL PORT

/dev/tty.usbmodem1101

Disconnect from port

SERIAL SETTINGS

TERMINAL MODE

WRITE TO FILE

> Type and press Enter to send

Send

```
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
Calculating the factorials of numbers from 1 to 10:
The factorial of  1 = 1
The factorial of  2 = 2
The factorial of  3 = 6
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
Calculating the factorials of numbers from 1 to 10:
The factorial of  1 = 1
The factorial of  2 = 2
The factorial of  3 = 6
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
```

CLEAR CONSOLE

USB-DEV
16BEDDB16139A2C3

TERMINAL SETTINGS ABOUT

SERIAL PORT

/dev/tty.usbmodem1101

Disconnect from port

SERIAL SETTINGS

TERMINAL MODE

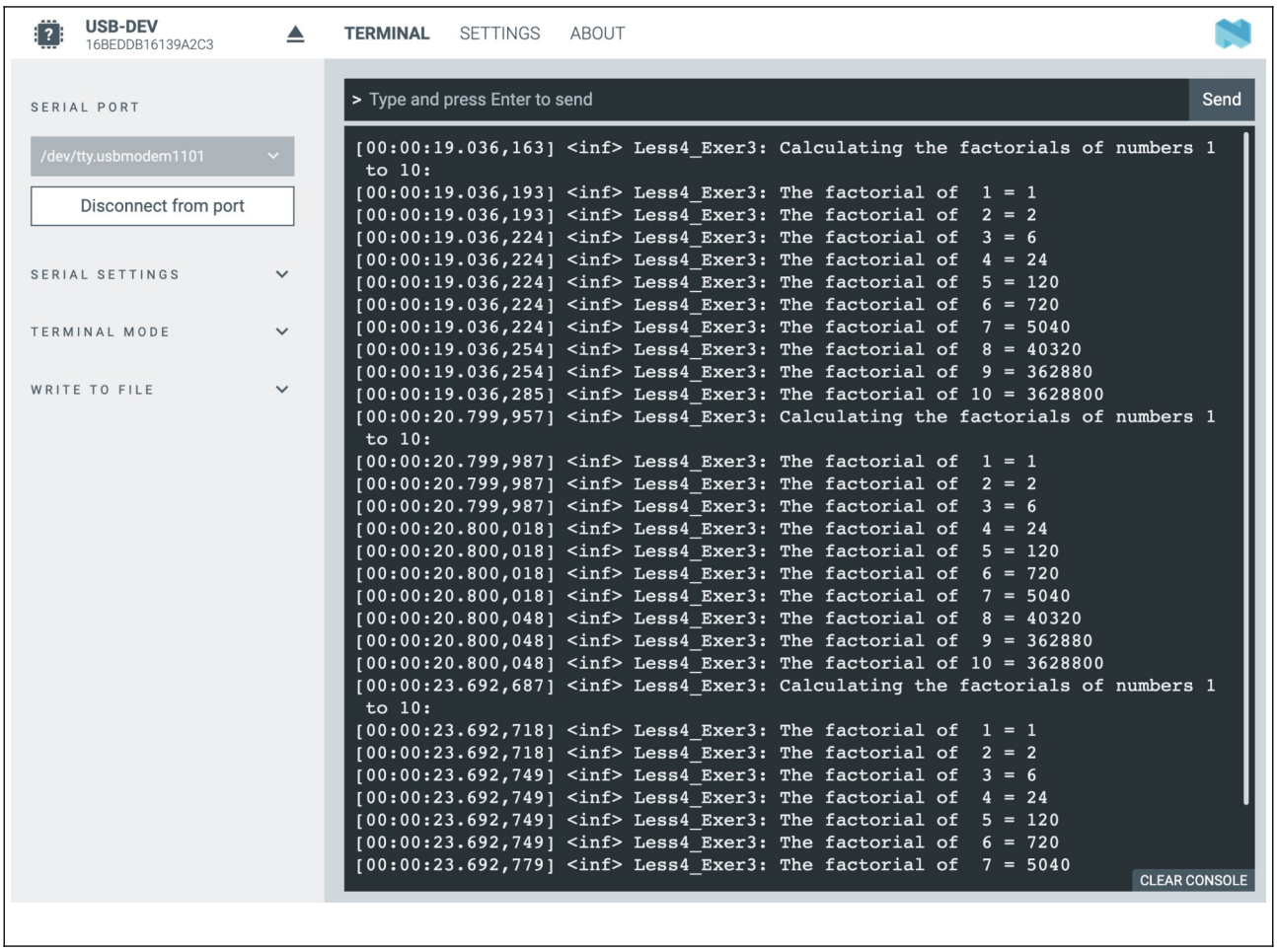
WRITE TO FILE

> Type and press Enter to send

Send

```
[00:00:00.000,854] <inf> Less4_Exer2: nRF Connect SDK Fundamentals
[00:00:00.000,854] <inf> Less4_Exer2: Exercise 2
[00:00:00.000,854] <dbg> Less4_Exer2: main: A log message in debug level
[00:00:00.000,854] <wrn> Less4_Exer2: A log message in warning level!
[00:00:00.000,885] <err> Less4_Exer2: A log message in Error level!
[00:00:00.000,885] <inf> Less4_Exer2: Sample Data!
                                      00 01 02 03 04 05 06 07  48 65 6c 6c 6f
          |........ Hello
[00:00:21.017,791] <inf> Less4_Exer2: Calculating the factorials of numbers 1
 to 10:
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  1 = 1
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  2 = 2
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  3 = 6
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  4 = 24
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  5 = 120
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  6 = 720
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  7 = 5040
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  8 = 40320
[00:00:21.017,883] <inf> Less4_Exer2: The factorial of  9 = 362880
[00:00:21.017,883] <inf> Less4_Exer2: The factorial of 10 = 3628800
[00:00:25.224,914] <inf> Less4_Exer2: Calculating the factorials of numbers 1
 to 10:
[00:00:25.224,945] <inf> Less4_Exer2: The factorial of  1 = 1
[00:00:25.224,945] <inf> Less4_Exer2: The factorial of  2 = 2
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  3 = 6
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  4 = 24
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  5 = 120
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  6 = 720
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  7 = 5040
[00:00:25.225,006] <inf> Less4_Exer2: The factorial of  8 = 40320
[00:00:25.225,006] <inf> Less4_Exer2: The factorial of  9 = 362880
[00:00:25.225,036] <inf> Less4_Exer2: The factorial of 10 = 3628800
```

CLEAR CONSOLE



Task

To filter the logger output and display only error messages, you need to adjust the logging system configuration at the project level.

Specifically, the logger's behavior can be controlled via the prj.conf configuration file, which allows you to override the settings defined in the source code. To achieve the desired filtering, the following directive has been added:

```
CONFIG_LOG_OVERRIDE_LEVEL=1
```

This configuration forces the global logger level to *error* , regardless of what is specified in the code via logging macros (such as LOG_INF, LOG_DBG, etc.). Consequently, all messages with a lower severity (informational or debug) are automatically ignored.

GenAI Statement

Generative AI tools were used to support study and writing during the workshops and in the preparation of this report.

In particular, these tools have been used to:

- improve the presentation quality of the report, making the text clearer, more structured and coherent;
- facilitate understanding of the technical documentation relating to the various course modules (e.g. nRF Connect SDK and Toolchain);
- support the translation and reformulation of content, in order to obtain more precise technical terminology;
- assist in solving practical problems, particularly related to compatibility between the system used (personal computer), the software tools and the board used in the laboratories.

All content generated with the support of GenAI tools has been critically reviewed, verified, and adapted based on personal understanding of the topics covered.